



KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN” YOGYAKARTA

JL. SWK 104 (Lingkar Utara) Condongcatur, Yogyakarta 55283 Telp.(0274)486188, 486733, Fax. 486400
Jl. Babarsari 2, Tambakbayan, Yogyakarta 55281 Telp.(0274)486911
Email : info@upnyk.ac.id. Homepage : <http://www.upnyk.ac.id>

KUIS MATA KULIAH PRAKTIKUM SEMESTER GASAL TA. 2026/2027
PRAKTIKUM IMPLEMENTASI STRUKTUR DATA
PERTEMUAN 05 - Stack (Tumpukan) dengan Linked List
KERJAKAN SESUAI NAMA FUNCTION PADA STUDENT.CPP

STUDY CASE

Anda diminta membuat bagian dalam sebuah aplikasi editor teks bernama Tulis. Jendelanya sudah jadi, menunya sudah lengkap, tetapi dua fiturnya masih mati total: tombol Undo tidak melakukan apa-apa, dan pemeriksa kurung selalu menjawab "tidak tahu".

Yang menarik, kedua fitur itu ternyata membutuhkan bentuk penyimpanan yang sama persis, dan itulah materi pertemuan ini: STACK.

Fitur Pertama

Setiap kali Anda mengetik sesuatu, editor mencatat perubahan itu. Ketika Anda menekan Ctrl+Z, yang dibatalkan adalah perubahan yang PALING TERAKHIR Anda lakukan, bukan yang paling awal.

Coba bayangkan kalau aturannya terbalik. Anda mengetik "halo", menghapus satu huruf, lalu menebalkan sebuah kata. Ctrl+Z seharusnya mengembalikan penebalan tadi. Kalau yang dibatalkan justru pengetikan "halo" yang paling awal, tulisan Anda akan berantakan.

Aturan "yang terakhir masuk, dialah yang pertama keluar" itu disebut LIFO (Last In, First Out). Bentuk penyimpanan yang mengikuti aturan itu disebut STACK, atau tumpukan. Namanya tumpukan karena persis seperti menumpuk piring: piring baru selalu diletakkan di ATAS, dan piring yang bisa diambil juga selalu yang paling ATAS, tanpa membongkar yang di atasnya.

Oleh karena itu riwayat perubahan disimpan sebagai Stack berbasis Linked List:

```
top
|
[30]  <- paling terakhir masuk, paling pertama keluar
|
[20]
|
[10]  <- paling pertama masuk, paling terakhir keluar
|
nullptr
```

`top` penunjuk ke elemen PALING ATAS. Bernilai `nullptr` berarti tumpukannya sedang KOSONG.

`next` penunjuk ke elemen DI BAWAHNYA. Elemen paling bawah `next`-nya bernilai `nullptr`.

Perhatikan: seluruh tumpukan hanya dikenali dari SATU penunjuk saja, yaitu `top`. Anda tidak pernah perlu tahu di mana dasarnya, karena semua operasi hanya terjadi di puncak.

Tumpukan ini dibangun dari linked list, jadi tidak ada batas banyaknya elemen dan tidak ada keadaan "penuh". Node dibuat dengan `new` saat data masuk, dan dilepas dengan `delete` saat data keluar. Yang tetap ada adalah keadaan KOSONG, dan itu keadaan yang sah, bukan kesalahan: menekan Ctrl+Z pada dokumen yang baru dibuka memang seharusnya tidak melakukan apa-apa.

Fitur Kedua

Tulis juga dipakai untuk menulis kode, jadi ia punya pemeriksa kurung: setiap `(`, `[`, dan `{` harus punya pasangan penutup yang sejenis, dan pasangannya tidak boleh bersilangan. Contohnya:

<code>(a + b) * (c - d)</code>	seimbang
<code>{ [()] }</code>	seimbang, tiga jenis bersarang rapi
<code>(a + [b])</code>	TIDAK — pasangannya bersilangan
<code>(a + b</code>	TIDAK — ada buka tanpa penutup
<code>) (</code>	TIDAK — penutup muncul lebih dulu

Sekilas ini soal yang sama sekali berbeda. Tetapi coba perhatikan barisan `{ [ ( ) ] }`. Ketika Anda bertemu `)`, yang harus dipasangkan dengannya adalah `(`, yaitu tanda buka yang PALING TERAKHIR dibuka dan belum tertutup. Sesudah itu, ketika bertemu `]`, pasangannya `[` yang sekarang menjadi yang terakhir belum tertutup.

Itu aturan LIFO yang sama persis. Jadi fitur kedua ini memakai tumpukan yang sama dengan fitur pertama, dan itulah sebabnya keduanya ada di satu study case.

FUNCTION YANG SUDAH DISEDIAKAN & TIDAK DINILAI		
<code>inisialisasi</code>	Menyiapkan tumpukan baru menjadi kosong	Inisialisasi
<code>isEmpty</code>	Apakah tumpukannya sedang kosong	Cek Kosong
<code>peek</code>	Melihat puncak tanpa mengambilnya	Peek
<code>display</code>	Membaca seluruh isi tumpukan menjadi satu baris teks	Traversal

Berikut satu sesi pemakaian Tulis. EMPAT langkah bertanda SOAL adalah pekerjaan yang harus Anda kerjakan.

1. Anda mengetik. Setiap perubahan dicatat ke tumpukan riwayat, dan yang baru selalu diletakkan di PUNCAK. **(SOAL 1)**
2. Status bar menampilkan perubahan terakhir tanpa membatalkannya, dan panel riwayat menampilkan seluruh tumpukan dari atas ke bawah. (Kedua fungsi untuk ini sudah disediakan, namanya *peek* dan *display*. Baca *peek* baik-baik, ia pembanding yang berguna untuk Soal 2.)

3. Anda menekan Ctrl+Z. Perubahan yang paling terakhir dibatalkan, dan editor perlu tahu perubahan apa itu supaya bisa mengembalikannya. Menekan Ctrl+Z pada dokumen yang belum diapa-apakan tidak boleh membuat aplikasi berhenti tidak wajar. **(SOAL 2)**
4. Anda menekan Ctrl+S. Dokumen tersimpan, dan seluruh riwayat undo dibuang sekaligus supaya tidak menumpuk di memori. **(SOAL 3)**
5. Anda beralih menulis kode. Editor memeriksa apakah tanda kurung yang Anda ketik sudah berpasangan dengan seimbang. **(SOAL 4)**

1. push (tambah di puncak)

25 Poin

Quest	Letakkan sebuah nilai baru di posisi PALING ATAS tumpukan. Seluruh nilai yang sudah ada tetap tersimpan dengan urutan yang sama persis, hanya saja sekarang mereka semua berada di bawah nilai baru itu. Sesudah pemanggilan, <code>s.top</code> harus menunjuk node yang baru, dan <code>next</code> miliknya harus menunjuk elemen yang tadinya berada di puncak. Lupa menyambung ini membuat seluruh isi lama hilang sekaligus bocor di memori. Tidak pernah ada penolakan karena penuh, jadi kembaliannya selalu <code>true</code> selama node barunya berhasil dibuat. Setiap pemanggilan menyediakan TEPAT SATU node baru.	
Parameter/ Variabel	s	Stack milik pemanggil. Bertanda <code>&</code> , sehingga perubahan pada <code>s.top</code> ikut terasa oleh pemanggil
	nilai	Nilai yang mau dimasukkan
	return	<code>true</code> bila nilai baru berhasil masuk
Gambaran	Sebelum : Top -> 20 -> 10 Operasi : <code>push(s, 30)</code> Sesudah : Top -> 30 -> 20 -> 10 (kembalian true) Tumpukan yang tadinya kosong: Sebelum : (kosong) Operasi : <code>push(s, 10)</code> Sesudah : Top -> 10 (kembalian true)	

2. pop (keluarkan dari puncak)

30 Poin

Quest	Keluarkan elemen PALING ATAS dari tumpukan, beri tahu pemanggil nilainya, lalu buang node-nya dari memori. Kembalian menyatakan berhasil atau tidaknya, sedangkan nilai yang keluar disampaikan lewat parameter <code>`nilai`</code> yang bertanda <code>`&`</code>. Bandingkan dengan <code>peek()</code> yang sudah disediakan: bedanya cuma satu, <code>peek</code> hanya melihat, sedangkan <code>pop</code> benar-benar mengeluarkan dan membuang node-nya. Isi <code>`nilai`</code> SEBELUM node di-<code>`delete`</code>, dan buang TEPAT SATU node per pemanggilan yang berhasil. Bila tumpukan kosong (underflow), kembaliannya <code>`false`</code> dan <code>`nilai`</code> TIDAK BOLEH DISENTUH SAMA SEKALI. Ini ikut diuji.	
Parameter/ Variabel	<code>s</code>	Stack milik pemanggil. Bertanda <code>`&`</code>
	<code>nilai</code>	Tempat penerima nilai yang keluar. Bertanda <code>`&`</code>
	<code>return</code>	<code>`true`</code> bila ada elemen yang benar-benar dikeluarkan, <code>`false`</code> bila tumpukannya sedang kosong
Gambaran	Sebelum : Top -> 30 -> 20 -> 10 Operasi : <code>int n; pop(s, n);</code> Sesudah : Top -> 20 -> 10, n bernilai 30 (kembalian true) Tumpukan kosong (underflow): Sebelum : (kosong) Operasi : <code>int n = -999; pop(s, n);</code> Sesudah : (kosong), n TETAP bernilai -999 (kembalian false)	

3. clear (kosongkan tumpukan)

20 Poin

Quest	Buang seluruh isi tumpukan sekaligus, sampai benar-benar kosong. Memutus sambungannya saja tidak cukup: <code>`s.top`</code> memang bisa langsung dibuat <code>`nullptr`</code>, tetapi seluruh node-nya masih menumpuk di memori tanpa ada yang bisa mencapainya lagi. Keadaan itu disebut kebocoran memori, dan itu ikut dinilai. SELURUH node harus dibuang dengan <code>`delete`</code> (banyaknya dihitung checker secara TEPAT), dan alamat node berikutnya harus disimpan SEBELUM sebuah node di-<code>`delete`</code>. Memanggilnya pada tumpukan yang sudah kosong, atau dua kali berturut-turut, harus aman. Sesudah dikosongkan, tumpukan tetap bisa dipakai lagi seperti biasa.	
Parameter/ Variabel	<code>s</code>	Stack milik pemanggil. Bertanda <code>`&`</code>
	<code>return</code>	Tidak ada (fungsi bertipe <code>`void`</code>)
Gambaran	Sebelum : Top -> 50 -> 40 -> 30 -> 20 -> 10 Operasi : <code>clear(s)</code> Sesudah : (kosong), s.top bernilai nullptr, dan kelima node sudah dibuang dari memori	

Quest	Tentukan apakah tanda kurung di dalam sebuah teks sudah berpasangan dengan seimbang. Ada tiga jenis yang diperiksa: `(` dengan `)`, `[` dengan `]`, dan `{` dengan `}`. Seimbang berarti setiap tanda buka punya penutup yang SEJENIS, setiap tanda tutup punya pembuka yang SEJENIS, dan pasangan-pasangannya tidak saling BERSILANGAN. Petunjuk: ketika bertemu sebuah tanda tutup, yang harus dipasangkan dengannya selalu tanda buka yang PALING TERAKHIR dibuka dan belum tertutup. Menghitung jumlah saja TIDAK CUKUP, yang menentukan adalah URUTANNYA. Di akhir pemeriksaan tidak boleh ada tanda buka yang masih menunggu pasangan. Karakter selain keenam tanda kurung itu DIABAIKAN, dan teks kosong bernilai seimbang. Boleh memakai <i>push</i> dan <i>pop</i> buatan sendiri, atau cara lain yang menghasilkan perilaku sama; bila memakai stack sendiri, bereskan node-nya sebelum fungsi selesai.	
Parameter/ Variabel	ekspresi	Teks yang mau diperiksa. Boleh sepanjang apa pun, boleh juga kosong
	return	<code>`true`</code> bila seimbang, <code>`false`</code> bila tidak
Gambaran	<pre> "(a + b) * (c - d)" -> true "{{[()]}" -> true tiga jenis, bersarang rapi "" -> true tidak ada kurung sama sekali "halo dunia" -> true karakter lain diabaikan "(a + b) * (c - d" -> false ada buka tanpa penutup "(a + [b)]" -> false pasangannya bersilangan ")(" -> false penutup muncul lebih dulu "[]" -> false penutupnya tidak sejenis </pre>	

CATATAN & ATURAN

- Hanya file ``src/student.cpp`` yang boleh diedit. ``src/student.h``, ``tests/checker.cpp``, dan ``tests/report.h`` tidak boleh diedit.
- Nilai yang disimpan bertipe ``int``. Nilai boleh negatif, boleh nol, dan boleh muncul lebih dari sekali.
- Tumpukan yang KOSONG adalah keadaan yang sah, bukan kesalahan.
- Tidak ada kapasitas dan tidak ada keadaan "penuh".
- Node dibuat dengan ``new`` dan yang keluar dilepas dengan ``delete``.
- Tidak ada satu pun fungsi yang mencetak ke layar.
- Signature fungsi serta bentuk ``struct Node`` dan ``struct Stack`` adalah kontrak; isi fungsi, nama variabel, dan struktur kode di dalamnya sepenuhnya bebas.
- Anda boleh menambah fungsi bantu sendiri.
- Soal 1, 2, dan 3 dinilai sendiri-sendiri: checker menyiapkan tumpukan ujinya tanpa memakai fungsi Anda.
- Soal 4 adalah PENERAPAN. Kalau Anda mengerjakannya memakai ``push`` dan ``pop`` buatan Anda sendiri, pastikan kedua soal itu sudah benar lebih dulu. Anda juga boleh mengerjakannya dengan cara lain, yang dinilai hanya hasilnya.
- ``cin`` hanya boleh dipakai DI DALAM ``main()``. Jangan menaruh ``cin`` di dalam keempat fungsi yang dinilai, karena checker memanggil fungsi-fungsi itu tanpa memberi masukan apa pun.